26B_CSE102T / Final Exam / CSE 102T Spring 2026 Final Exam VPL

CSE 102T Spring 2026 Final Exam VPL

Description Submission view

Available from: Çarşamba, 20 Mayıs 2026, 9:00 AM
Due date: Çarşamba, 20 Mayıs 2026, 10:35 AM
Requested files: OBS.java (Download)
Type of work: Individual work

OBS Simulation

We will be simulating the OBS system. To assist, you will need to implement a Grades class that takes three Generic types. The first is the type for Student objects, the second is the Course type, and the third is the Grade type. This allows the same class to be used in different ways — for example, identifying students by name or student IDs or using numeric scores or grade letters for grades.

Methods:
1. addStudent(student) — Registers a student in the system with no courses. Though, if the student already exists, does nothing (i.e. does not remove them from any courses). Returns none.
2. addCourse(student, course) — Enrolls a student in a course with no grade assigned yet. Automatically adds the student to the system if not already registered. Returns none.
3. setGrade(student, course, grade) — Assigns or updates a grade for a student in a course. Automatically adds the student to the system and course if not already registered. Returns none.
4. getGrade(student, course) — Returns the grade for a specific student and course. Throws StudentNotFoundException if the student is not in the system, or CourseNotFoundException if the student is not enrolled in that course. Returns the type for grade based on the Grades object creation.
5. getStudents() — Returns a Collection of all students in the system. The collection must contain objects of the type of Student.
6. getStudents(course) — Returns a Collection of all students enrolled in a given course. The collection must contain objects of the type of Student.
7. getCourses() — Returns a Collection of all courses currently taken by any student in the system. The collection must contain objects of the type of Course.
8. getCourses(student) — Returns a Collection of all courses a given student is enrolled in. Throws StudentNotFoundException if the student does not exist. The collection must contain objects of the type of Course.
9. transcript(student) — Returns a formatted string listing all courses and grades for a given student.
10. toString() — Returns a formatted string containing the transcripts of all students in the system.

Exceptions
1. StudentNotFoundException — Exception for when a student does not exist. toString method returns "StudentNotFoundException: Student " + student + " not found in our university".
2. CourseNotFoundException — Exception for when a course does not exist for a given student. toString method returns "CourseNotFoundException: Course " + course + " not found for Student " + student

NOTE: grades can be null — a student can be enrolled in a course before a grade has been assigned. In this case the getGrade and transcript methods may return/use the null values with no error.

IMPORTANT NOTE: If you change any method that is already correctly implemented, your score will be significantly reduced.

Requested files

OBS.java

```java
import java.util.HashMap;
import java.util.HashSet;
import java.util.Set;
import java.lang.RuntimeException;

public class OBS {
   public static void main(String[] args) {

      Grades<String, Integer, String> akdenizName = new Grades<>(); // grades by name (Joseph LEDET), courseco

      akdenizName.addStudent("Will Byers");
      akdenizName.addStudent("Mike Wheeler");
      akdenizName.addStudent("Eleven Hopper");
      akdenizName.addStudent("Dustin Henderson");
      akdenizName.addStudent("Lucas Sinclair");
      akdenizName.addStudent("Max Mayfield");

      akdenizName.setGrade("Will Byers", 101, "BA");
      akdenizName.setGrade("Will Byers", 102, "AA");
      akdenizName.setGrade("Mike Wheeler", 101, "BB");
      akdenizName.setGrade("Mike Wheeler", 102, "FF");
      akdenizName.setGrade("Eleven Hopper", 201, "AA");
      akdenizName.setGrade("Eleven Hopper", 204, "BA");
      akdenizName.setGrade("Dustin Henderson", 102, "CC");
      akdenizName.addCourse("Dustin Henderson", 201); // enrolled but has not earned a grade yet
      akdenizName.setGrade("Lucas Sinclair", 202, "BC");
      akdenizName.setGrade("Max Mayfield", 101, "FF");
      akdenizName.addCourse("Max Mayfield", 102); // enrolled but has not earned a grade yet

      System.out.println("Will Byers' courses: " + akdenizName.getCourses("Will Byers"));
      System.out.println("Will Byers' 101 grade: " + akdenizName.getGrade("Will Byers", 101));
      System.out.println("All grades for Dustin:\n" + akdenizName.transcript("Dustin Henderson"));
      System.out.println("Students in CSE101: " + akdenizName.getStudents(101));
      System.out.println("All courses offered: " + akdenizName.getCourses());

       // Exception examples
      try {
         akdenizName.getGrade("Joyce Byers", 101); // student not in system
      } catch (RuntimeException e) {
         System.out.println(e);
      }

      try {
         akdenizName.getGrade("Will Byers", 201); // course not taken by this student
      } catch (RuntimeException e) {
         System.out.println(e);
      }

      System.out.println(akdenizName);
      akdenizName.setGrade("Max Mayfield", 102, "BB"); // now earned a grade in 102
      System.out.println(akdenizName);

      // We can also store grades as double using student ID's and course codes that include department ID
      Grades<Long, String, Double> akdenizID = new Grades<>(); // grades by student ID (1234), coursecode (CSE

      akdenizID.setGrade(20200808001L, "CSE101", 72.5);
      akdenizID.setGrade(20200808001L, "MAT101", 91.0);
      akdenizID.setGrade(20210809001L, "CSE101", 65.0);
      akdenizID.setGrade(20210809001L, "HIS101", 78.3);
      akdenizID.setGrade(20190808123L, "AIE201", 95.0);
      akdenizID.setGrade(20190808123L, "CSE204", 88.0);

      System.out.println("Score for 20200808001 in MAT101: " + akdenizID.getGrade(20200808001L, "MAT101"));
      System.out.println("All grades for 20210809001:\n" + akdenizID.transcript(20210809001L));

       // Exception example
      try {
         akdenizID.transcript(1234L); // student does not exist
      } catch (RuntimeException e) {
         System.out.println(e);
      }
      additionalMain();
   }
   // Do not modify anything above this line

   public static void additionalMain() {
       // you may use this method for additional testing
   }
}

class Grades {
   private HashMap<S, HashMap<C, G>> map = new HashMap<>();

   public void addStudent(S student) {
      if(!map.containsKey(student))
         map.put(student, new HashMap<>());
   }

   public void addCourse(S student, C course) {
      if(!map.get(student).containsKey(course))
         map.get(student).put(course, null);
   }

   public void setGrade(S student, C course, G grade) {
   }

   public G getGrade(S student, C course) {
      try {
         if(map.containsKey(student))
            if(map.get(student).containsKey(course))
               return map.get(student).get(course);
            else
               throw new CourseNotFoundException(student, course);
         else
            throw new StudentNotFoundException(student);
      } catch(Exception e) {
         System.out.println("Vecna is here.");
      }
   }

   public Set<S> getStudents() { // get all students in the university
      return map.keySet();
   }

   public Set<C> getCourses(S student) { // get courses taken by a particular student
      if(map.containsKey(student))
         return map.get(student).keySet();
      throw new StudentNotFoundException(student);
   }

   public Set<C> getCourses() { // get all courses currently taken by any student
      Set<C> set = new HashSet<>();
      int[] arr = new int[0];
      arr[0] = 0;
      for(S student: map.keySet())
         for(C course: map.get(student).keySet())
            set.add(course);
      return set;
   }

   public Set<S> getStudents(C course) { // get students in a particular course
      return null;
   }

   public String transcript(S student) {
      if(!map.containsKey(student)) {
         String ret = "Transcript for Student " + student + ":\n";
         for(C course: map.get(student).keySet())
            ret += "\t" + course + ": " + map.get(student).get(course) + "\n";
         return ret;
      }
      throw new StudentNotFoundException(student);
   }

   public String toString() {
      String ret = "All students transcripts\n";
      for(S student: map.keySet())
         ret += "\n" + transcript(student) + "\n";
      return ret;
   }
}

class StudentNotFoundException {
   private Object student;
   public StudentNotFoundException(Object student) {
      this.student = student;
   }

   public String toString() {
      return "StudentNotFoundException: Student " + student + " not found in our university";
   }
}

class CourseNotFoundException {
   private Object student, course;
   public CourseNotFoundException(Object student, Object course) {
      this.student = student;
      this.course = course;
   }

   public String toString() {
      return "CourseNotFoundException: Course " + course + " not found for Student " + student;
   }
}
```